Robotics Project

Roann CANTEL Diego FLEURY CORREA DE MORAES
Sergio MAGALHÃES CONTENTE Seydou SENE Ada YETIS

April 2026

1 Introduction

In this report, we present our results for the **Open Electric Kettle Lid** atomic task in the *RoboCasa* simulator, using the **PandaOmron** robot. We explored **Imitation Learning** via two variants of Behavioral Cloning (BC) with reward shaping as our primary method. Both were trained offline on 50 human expert demonstrations from the RoboCasa `pretrain/human` dataset (110 episodes available, 5,146 transitions used, avg. episode length 103 steps), then evaluated in the simulator.

Unfortunately, neither approach achieved task success in evaluation. The reasons are detailed below.

2 Method: Two BC Variants

Simple BC (MSE). A plain supervised MLP (`BCPolicy`) trained by minimizing the mean squared error between predicted and expert actions:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|\pi_\theta(s_i) - a_i^*\|^2$$

Imitation BC (NLL). An `ActorCriticPolicy` (Stable-Baselines3) trained via the `imitation` library, which maximizes the log-likelihood of expert actions under the stochastic policy, adding an entropy bonus and optional L2 regularization:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\log \pi_\theta(a^* \mid s) - \lambda_H\, H(\pi_\theta(\cdot \mid s)) + \lambda_{L2}\|\theta\|_2^2$$

with $\lambda_H = 10^{-3}$ and $\lambda_{L2} = 0$.

Reward shaping

The reward shaping strategy was designed to make the reward landscape denser over the state space. It combines a distance-based reaching term, a lid-opening progress term, and a task success bonus. The distance between the end-effector and the kettle is denoted by $d = \|\mathbf{p}_{eef} - \mathbf{p}_{kettle}\|$. This distance is normalized to the range $[0, 1]$ using a hyperbolic tangent function to obtain the reaching reward. Additionally, a reward proportional to the lid opening progress is included to encourage approaching the kettle from the correct side and angle. Finally, a sparse success bonus is added when the task is completed. The resulting dense reward is defined as:

$$R_{\text{dense}} = w_r\,(1 - \tanh(\alpha\|\mathbf{p}_{eef} - \mathbf{p}_{kettle}\|)) + w_l \cdot \text{lid} + \begin{cases} B, & \text{if success} \\ 0, & \text{otherwise} \end{cases}$$

Training Results

Both models converged during training, though their loss scales are not directly comparable (MSE vs. NLL). Simple BC reduced its MSE loss from 0.1052 to 0.0057 (a 94.6% reduction). Imitation BC reduced its neglogp loss from 12.20 to 4.19 over 1,900 batches, while `prob_true_act` grew from 5.26×10^{-6} to 1.51×10^{-2} ($\times$2,870). Training curves are shown in Appendix A.

Despite training convergence, both policies achieved **0% success rate** across 20 rollout episodes, with every episode timing out at 500 steps as per table 1.

3 Analyses and Conclusion

The core difficulty was the gap between supervised training convergence and real task performance. This was partially expected as pure behavioral cloning models are not very robust and suffer from *covariate shift*. Since the starting conditions in the evaluation were not identical to the training set, the robot could not apply the very narrow methodology it had learned for success, any small action error at test time moves the robot into states outside the training distribution, and the policy has no mechanism to recover. The high-dimensional continuous action space and precise manipulation required (fine-grained lid opening) exacerbate this effect.

As visible in the evaluation plots (Appendix B), both policies yield zero lid progress across all 20 episodes, with only negligible non-zero rewards appearing sporadically in a handful of runs - confirming the robot never meaningfully interacts with the kettle. Notably, Imitation BC shows slightly more non-zero reward events at later timesteps (∼step 300), suggesting its stochastic policy explores more than the deterministic MSE variant, yet still falls far short of task completion.

However, between the two approaches, Imitation BC (NLL) is the more principled and extensible approach, as its ActorCritic architecture is directly compatible with subsequent PPO or GAIL fine-tuning, which we identify as the essential next step. If we were to continue experiments, we could overcome these problems by combining BC pre-training with online RL fine-tuning (e.g., PPO with better **denser** reward shaping), or use iterative approaches such as DAgger, for more directly correcting the distribution shift.

4 Other Approaches

In parallel, we also explored **training PPO from scratch** (without BC pre-training) using a **curriculum learning** scheme with 3 stages of increasing difficulty: Stage 0 (fixed position, no randomization), Stage 1 (random layout, fixed style), and Stage 2 (full scene and texture randomization). After inspecting the task source code, we found that success requires pressing a small button with only 2 mm of travel, which triggers automatic lid opening (`lid` ≥ 0.95). A **different dense reward was designed** with 4 components:

$$R_{\text{RL}} = \underbrace{10 \cdot \mathbf{1}[\text{success}]}_{\text{sparse}} + \underbrace{5.0 \cdot \text{lid_val}}_{\text{lid progress}} + \underbrace{3.0 \cdot \frac{\text{button_val}}{0.1}}_{\text{button depression}} + \underbrace{\exp\left(-\frac{\|\mathbf{p}_{eef} - \mathbf{p}_{btn}\|^2}{2\sigma^2}\right)}_{\text{Gaussian approach}}$$

where $\mathbf{p}_{btn}$ is the button position extracted from the MuJoCo XML model.

Two PPO runs of ∼500k steps each were performed on an L40S GPU (∼5h/run). The second run used a wider network (256×256), `log_std_init`$= -1.0$, `ent_coef`$= 0.001$, and batch size 256. While training reward climbed above 500, `explained_variance` remained ∼0, indicating the value function failed to model returns. The policy stayed too stochastic (`std` ~ 2.0) for the required precision. Final result: **0% success** on 50 evaluation episodes.

We further explored behavioral cloning with **action chunking**. Action chunking essentially forces the model to decide on its next steps in chunks so instead of predicting the next move, it predicts the next 10, runs through them, and then reevaluates. We hoped that this would at least lessen the impact of the covariate shift. We also augmented the training dataset. Although the model still did not perform well on evaluation (only succeeding 10% of the time), comparing the videos of the trained robot's attempts at opening the kettle lid, we can see that with this training strategy, the robot arm actually found the kettle and tried the area around it to try to open the lid.

We finally attempted to train a **GAIL policy** using the GPU provided by the school servers. However, we ran into several computational limitations. The time allocated to students was not enough for us to train the model without vectorizing the environment but the vectorized environment led to deadlock issues and out of memory errors due to the size of the camera tensors. It would be interesting to see how much the model could have improved if we had had the computational resources to be able to train this policy.

As a separate attempt 6, we also ran GAIL on Colab (single environment, no vectorisation) to sidestep the deadlock issues encountered on the school servers. Training ran for 48 rounds (∼98k timesteps) with a PPO generator against a learned discriminator. However, the discriminator's wrapped reward decreased monotonically from 327 to 84.9 over training, indicating the discriminator grew increasingly confident at rejecting the generator's rollouts rather than the generator improving. Evaluation over 20 episodes still yielded 0% success with every episode timing out, matching the BC results. The single-environment setup (∼20 FPS) proved too slow for meaningful convergence within Colab's session limits.

A Training Curves

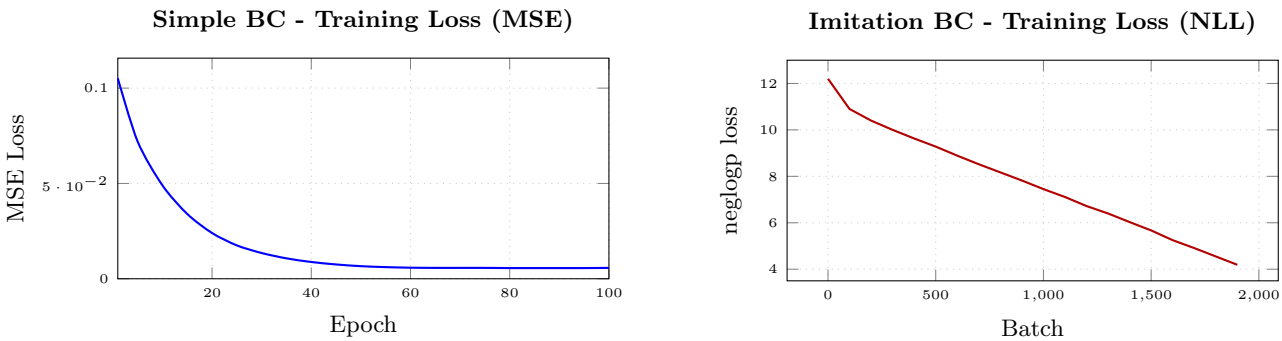

Figure 1: **Left**: Simple BC MSE loss per epoch. **Right**: Imitation BC neglogp loss per batch.

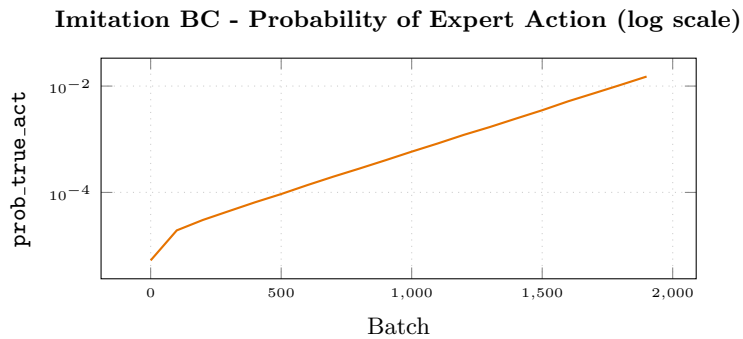

Figure 2: Probability the Imitation BC policy assigns to the expert action

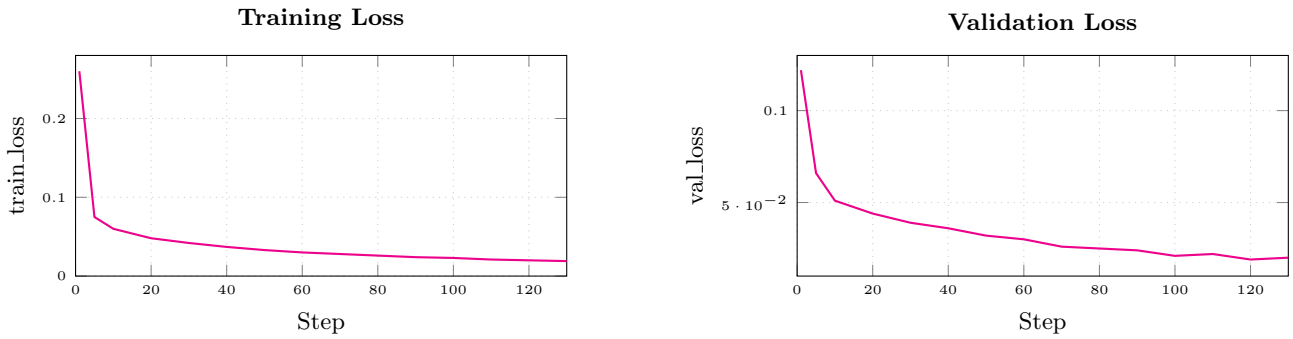

Figure 3: **Left**: Training loss per step for chunked imitation. **Right**: Validation loss per step for chunked imitation.

B Evaluation Plots

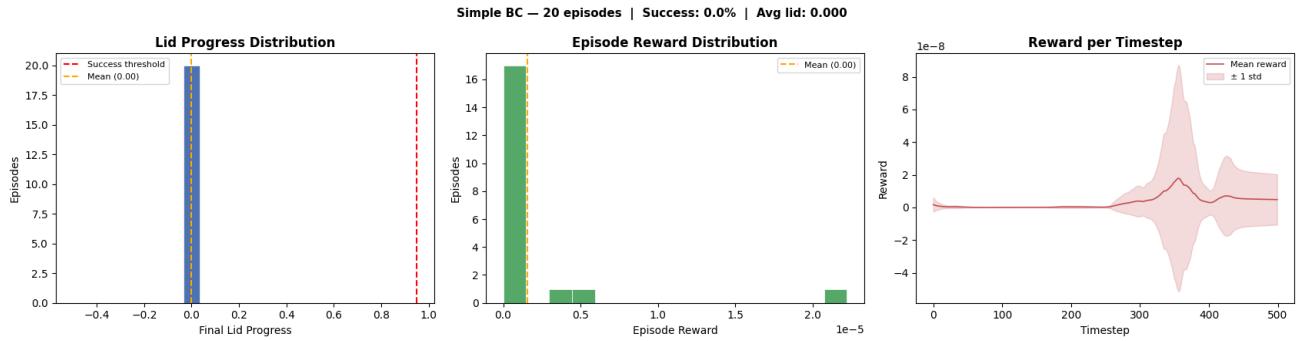

Figure 4: Simple BC evaluation (20 episodes): lid progress distribution, episode reward distribution, and mean reward per timestep. All episodes time out with zero lid progress; the deterministic policy shows near-zero reward throughout, with larger variance appearing only after step 300.

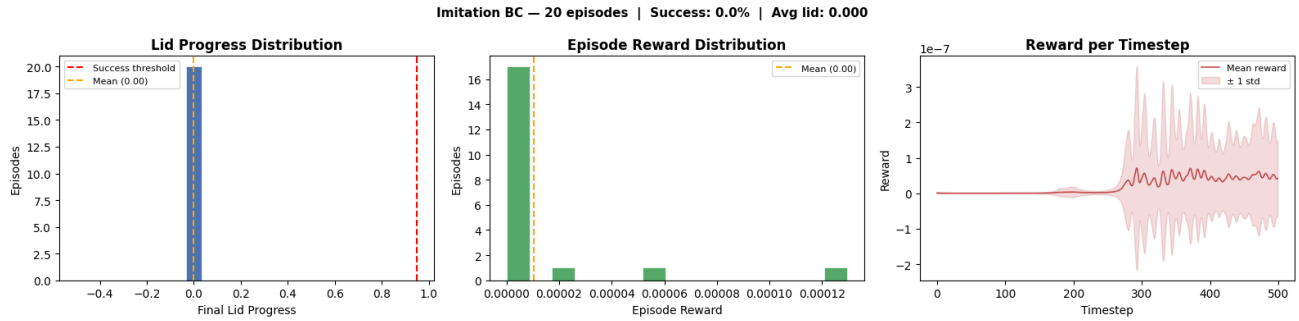

Figure 5: Imitation BC evaluation (20 episodes): lid progress distribution, episode reward distribution, and mean reward per timestep. All episodes time out with zero lid progress; sparse non-zero rewards appear around step 300, reflecting the stochastic policy's occasional exploration.

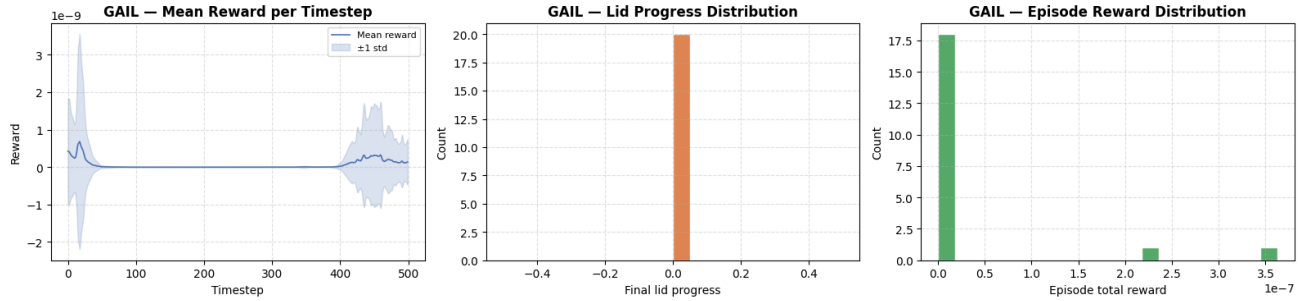

Figure 6: GAIL evaluation (20 episodes): lid progress distribution, episode reward distribution, and mean reward per timestep. All episodes time out with zero lid progress. The true environment reward is negligible ($\sim 10^{-9}$), with slight variance bursts in the first 50 steps and around step 400, confirming the policy never meaningfully interacts with the kettle.

C Results

Table 1: Evaluation results (20 episodes, deterministic rollout)

Metric	**Simple BC (MSE)**	**Imitation BC (NLL)**
Success rate	0.0%	0.0%
Avg. lid progress	0.000	0.000
Avg. episode reward	0.000	0.000
Avg. episode length	500	500
Training loss (initial $\rightarrow$ final)	$0.1052 \rightarrow 0.0057$	$12.20 \rightarrow 4.19$
Loss reduction	94.6%	65.7%